# Project Rules

Proff. Alessio Martino, Lorenzo Ariemma & Mattia Cervellini

February 17, 2026

## 1 Preliminary Instructions on Project Submission

### 1.1 When?

You must submit all the final material for your project by May $10^{\text{th}}$, 2026. At 23:59 Rome time, we will clone your repository locally and this will be your submission.

### 1.2 What?

Each group must populate a private git-based repository with the following materials:

1. The presentation discussed during the Checkpoint slot: a `.pdf` file called `checkpoint_presentation.pdf` and, if available, a `.pptx` file called `checkpoint_presentation.pptx`;
2. The presentation discussed during the Pitch slot: a `.pdf` file called `pitch_presentation.pdf` and, if available, a `.pptx` file called `pitch_presentation.pptx`;
3. A folder called `src` with the working code (more on this under Source Code Guidelines);
4. A .pdf file called `technical_report.pdf` containing a technical report about your work (more on this under Technical Report Guidelines);
5. Any additional file with the results, properly formatted (if requested by the company).

We will be pre-checking those files with an automatic script, so make sure to comply with the rules. Further, please note that this material will be shared with the respective companies for their evaluation.

### 1.3 How?

Every group has been asked to provide a team name: that name is also the name of the private git repository that you will be using to host and maintain your project. All repositories have been created already under a common `DASA-25-26` GitHub organization. Make sure to fill the corresponding form[1] asking for the GitHub username to get access to the repository and start working/collaborating with your peers.

## 2 Source Code Guidelines

Your code must be executable in its entirety and must guarantee the reproducibility of your experiments (including figures and numerical results). The `src` folder of your project should contain multiple files but still be ordered. We expect the code to be divided into multiple Python files by their role (e.g., `preprocessing.py`, `modelling.py`, etc. . . ). Each one of these files should be self-contained and contain the working code described by the file name. Each file should then have a "header" of comments inside of it describing what is expected to be found inside the file and the role in the whole project of that file. To let us understand what is the right order of execution of such files, we suggest to write a `README.MD` file inside the `src` folder, containing a small and minimalistic how-to guide on how to run your code[2].

The source code must be clean and clear: comments (and/or markdown cells in case of Notebooks) are more than welcome[3]. The code files should not have useless functions (functions declared but not used) or whole sections of the code commented out. We will not take into consideration the commented code and you'll lose points on code readability.

To ensure reproducibility, you are required to add a detailed list of all the libraries you have been using, their version[4] and the Python version.

## 3 Technical Report Guidelines

The technical report must a PDF file of at most 15 pages. It must include each of the following portions (unless marked as "optional"). Also here, the page limits and the structure of the report will be pre-checked with an automatic script.

### 3.1 Title page (1 page)

- Title of your project work (be creative!)

[1] See the latest announcement.

[2] This additional how-to guide has not to be confused with the Technical Report and shall not be part of it.

[3] However, do not use too many comments, otherwise your code will be unreadable.

[4] Use the Python Requirements standard file format for that.

- Name of the team
- Team members name and e-mail addresses (mark who is the group delegate)
- Name of the company

### 3.2 Main document body (max. 12 pages)

**Section 1: Introduction** Briefly describe your project, the case study, the goal, and the data you have been working with. Feel free to anticipate the results you have obtained and the techniques you have been using, but do not be too specific or too technical[5].

**Section 2: Methods** Describe the process of data collection (if applicable), pre-processing, (exploratory) analysis, training and testing. Some common aspects of this section include:

1. Data collection: which data you have been collecting and why;
2. Data pre-processing: starting from the raw data (either provided by the company or collected autonomously) what sorts of pre-processing, cleaning, missing value imputation, training/testing splitting (and so on) have you been carrying on and why;
3. Which models you have been considering and why;
4. Did you perform any hyperparameter tuning? If so, how?
5. Which performance indices and evaluation metrics have you been using and why;
6. Did you choose any baseline models to compare your models? If so, which one(s)?

Bear in mind that an external reader should understand why you made your design decisions (e.g., based on your EDA or other preliminary analyses or comparative analyses), how you cleaned the dataset and the reasons behind any other choices related to the project. To better guide the reader towards understanding your data science pipeline we strongly recommend to include figure(s) – e.g., flowcharts or block diagrams – to describe the proposed solution.

**Section 3: Results and Discussion** Describe all results you have been obtained. Given the nature of the projects, you are required to present two different "families" of results:

1. Technical results obtained by your analysis (i.e., the steps presented in Section 2)

[5] There will be separate sections to discuss those. Keep reading.

2. Project-oriented results, i.e., the business value-related implications of your outcomes (according to the desiderata by the company).

**Section 4: Conclusions** This section concludes your work and lists some remarks, notably:

1. A summary of what is (are) the take-away point(s) from your work;
2. Explain what questions may not be fully answered by your work as well as natural next steps from the direction of future work.

## 3.3 Bibliography [optional]

Include any bibliography or reference needed.

## 3.4 Appendices

**Appendix A: Code Description (max. 1 page)** Describe your code at a high level so that a reader could easily follow up your implementation. Do not copy-and-paste your full code or code snippets, use pseudocodes and/or flowcharts instead.

**Appendix B: Author Contribution and Generative AI Statement (max. 1 page)** A peer self-evaluation on who-did-what in the project. Please use the standard CRediT statement to write the author contribution paragraph. If one (or more) of the terms does not apply to your project (e.g., Funding acquisition), please ignore it. For the Generative AI Statement, you are required to state which Generative AI models have been used, to which extent, and for which purpose (see Section 7). If you have not been using any Generative AI tool, please state "In this work, we have not employed any GenAI tools."

*Important note #1*: you are free to include as many tables and figures as you want, but bear in mind the page limits, so choose what to represent (and how) carefully. Every table and every figure must be accompanied by a caption and a sequential numbering and must be referred to in the text. Same goes with the bibliography: each bibliographical entry must be numbered and cited in the text.
*Important note #2*: Sections 2 and 3 are very likely to be quite long and complex as they contain the vast majority of your work. Feel free to separate them into sub-sections (numbered 2.1, 2.2, and so on, respectively, and 3.1, 3.2, 3.3, and so on, respectively) at your convenience. The titles of the four "main" Sections are given, but you are free to choose the title for each of the sub-sections (depending on how you plan to hierarchically organise the contents, the titles must be meaningful). Section 1 and 4 (Introduction and Conclusions) normally are not that long and self-contained but if you really need, you can split them as well into sub-sections.

# 4 Final Evaluation

Each student will be evaluated according to the following criteria:

- at most 10 pts will be given for the theoretical test (i.e., 30% of the final grade);
- at most 4 pts will be given for a comprehensive usage of the git platform (as explained in Section 5);
- $\pm 2$ pts will be given for the peer review activity (explained in Section 6);
- at most 14 pts will be given for: quality of presentations, code quality, technical report, and project results;
- at most 1 bonus pt will be given for the homeworks completed during Winter break;
- at most 1 bonus pt will be given for the in-class challenge held on 06/02/2026.

**Note #1** the maximum total score from graded components is 32, this because homework and challenge points are treated as bonus points. Any laude will be awarded at the professor's discretion and is not determined directly by the numerical grade.

**Note #2** the Intermediate Pitch will not be subject to evaluation. However, it's a great opportunity for you to ask questions and get feedback. Yet, the slides must be submitted as well.

# 5 Usage of the Git platform

Git should be used as a collaboration and version control tool.

Each group member is expected to contribute meaningfully to the shared repository. Individual contributions must be visible through commits. Large, infrequent commits are discouraged; instead, students should commit small, logical changes often.

Every commit must include a clear and descriptive message explaining what was changed and why (e.g., "Implement user login validation" rather than "update" or "fix").

The repository should remain organized and readable. Temporary files, build artifacts, and IDE-specific files must not be committed; a proper `.gitignore` file should be used. Meaningful directory structures and consistent naming conventions are expected (various naming conventions are explained in Sections 1.2 and 2).

Create inside the `src` folder a `io` folder that will not be tracked on Git where you will store output files (even intermediate results if you need to) and from

which you will read the input files (we will put the files there before running your code). Do not create other files at execution time outside of that folder.

Commit history will be reviewed as part of the assessment. Evidence of collaboration, and balanced participation will positively impact the evaluation. Conversely, poor practices such as vague commit messages, committing generated files, last-minute bulk commits, or having one member perform most of the work may negatively affect grades.

# 6 Peer Review

Each student must evaluate two randomly-assigned groups[6]: (I) one will be on the same company as the student's one; (II) the other one will be on another company.

The peer review will be submitted on a proper form on MyLuiss and has to be provided by May $10^{\text{th}}$, 2026. Professors will evaluate each review and, if properly assessed, will award points according to the following three scenarios:

**Critical Review:** a review is considered to be critical if the reviewer finds a flaw in the methodology or in the technical approach of the presenting group. In case the group being reviewed (a.k.a., reviewee) receives a critical review, the reviewer will gain 1 point and $1/3^{\text{rd}}$ of a point will be taken away from each student belonging to the reviewee group;

**Validation Review:** a review is considered to be validating if the reviewer does not identify substantive flaws in the methodology or technical approach, and it offers a clear, reasoned justification of why the approach appears sound, highlights specific strengths of the work, and demonstrates critical engagement with the presentation (e.g., by discussing design choices, assumptions, or limitations acknowledged by the presenters). A validation review will be accepted only if no other reviewers or the Professors identify relevant flaws that were reasonably detectable from the presentation. Generic praise or unsupported statements will not be considered sufficient (hence, blank reviews). In case of validating reviews, if confirmed by the Professors, the reviewer still gains 1 point and the group will not loose any points for their work. Conversely, in case a positive review is not validated by the Professors or other reviewers, the reviewer will loose 1 point;

**Blank Review:** Any reviews left blank will be awarded 0 points to the reviewer.

The peer review will be based only on the pitch presentation: reviewers will not have access to the submitted materials (i.e., code and/or technical report). In order to facilitate the review process, students are allowed to ask questions to the presenting group during the final pitch presentation if something is not clear.

[6] Assignments will be made available in late April privately to each student.

**Structure of a review** For each review, we expect at most 1500 characters (including spaces) of commentary. We encourage the usage of numbered lists to separately describe your highlights.

**Availability of the reviews** Reviews can be made available, upon request, properly anonymised. The group delegate of the interested group can make a written request via e-mail after the deadline.

**Note** One might notice that the worst case scenario for a student could lead to loosing more than 2 points. However, as anticipated in Section 4, the maximum penalty will be 2 points.

# 7 On the usage of Generative AI tools

We strongly discourage the blind usage of any Generative AI tool because of the following reasons:

- copying text or code you do not fully understand undermines learning objectives and makes your work fragile;
- LLM outputs can be wrong, outdated, or inconsistent while still sounding confident (hallucinations).

Under a more pedagogic viewpoint, it promotes shallow understanding instead of deep mastery of concepts and techniques.

Nonetheless, we accept a responsible use of AI for brainstorming, outlining or clarifying concepts. All AI-generated content must be reviewed, understood, tested, and validated:

- Always run, test, and validate generated code.
- Cross-check factual statements with reliable sources.
- Prefer using AI for guidance and alternatives rather than final answers.

Students will take full responsibility of all submitted materials. Please note that during your presentations and/or during the peer-review activities you might be asked to explain design choices, models, metrics, and code structure. Inability to explain a component indicates lack of ownership, which can be sourced to cheated or plagiarised solutions. In summary, all submitted materials (e.g., code, report, slides, etc...) must be fully understood by each student.